

ПРОГРАММНЫЙ КОМПЛЕКС АН-MEMORYHUB
АН-MEMORYHUB

Пояснительная записка

(электронный документ)

Листов 12

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

АННОТАЦИЯ

1. Объект разработки:

Объект разработки — прототип гибридного нейросимвольного агента АН-MemoryHub для анализа текстовых журналов инцидентов. Цифровой двойник оператора АСУ ТП — целевой сценарий применения; промышленное внедрение и работа с реальным оператором данной запиской не подтверждаются.

2. Цель:

Цель — проверить гипотезу о том, что АГ-память с типизированными актантными ролями и тактовой активацией позволяет восстанавливать связь ответа с фактами и исходным текстом. Сравнительное преимущество перед контрольным векторным RAG подлежит измерению по М4.

Рассмотрены формальная модель памяти, механизм активации и распространения состояния по ассоциативным связям, приём фактов из текста с указанием происхождения и подтверждения, а также архитектура программной системы и её основные алгоритмы.

3. Ключевые слова:

Нейросимвольный ИИ, Ассоциативно-гетерархическая память, актантные роли, цифровой двойник оператора АСУ ТП, объяснимый ии, граф знаний, NEO4J, импульсное распространение активации.

Примечания:

1. Ссылка на GitHub репозиторий проекта: <https://github.com/Hausmaster333/АН-MemoryHub>

СОДЕРЖАНИЕ

1. Введение	3
2. Назначение и область применения	4
2.1. Назначение программы	4
2.2. Область применения	4
3. Технические характеристики	5
3.1. Постановка задачи на разработку программы	5
3.2. Описание применяемых математических методов, допущений и ограничений	5
3.3. Описание алгоритма и функционирования программы	6
3.3.1. Общая архитектура	7
3.3.2. Алгоритм приёма фактов из текста	7
3.3.3. Алгоритм активации	8
3.3.4. Сборщик мусора	9
3.3.5. Интерпретатор предметно-ориентированного языка	9
3.3.6. Взаимодействие с внешними системами	9
3.4. Обоснование выбора метода организации входных и выходных данных	9
3.5. Описание и обоснование выбора состава технических и программных средств	10
4. Ожидаемые технико-экономические показатели	11
5. Источники, использованные при разработке	12

1. ВВЕДЕНИЕ

Тема разработки — программный комплекс АН-МетодуHub: гибридный нейросимвольный ИИ-агент на основе ассоциативно-гетерархической (АГ) памяти, реализующий цифрового двойника оператора АСУ ТП для работы с журналами инцидентов.

Разработка ведётся на основании задания хакатона «Воспламенение 1.0» и формальной модели

ассоциативно-гетерархической памяти $АН = \langle S, C, P, H, L \rangle$, изложенной в монографии Романа Викторовича Душкина.

2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ

2.1. Назначение программы

Программа АН-MemoryHub реализует гибридного нейросимволического ИИ-агента на основе ассоциативно-гетерархической памяти. Агент предназначен для приёма и структурной обработки журналов инцидентов технологического процесса, построения цепочек зависимостей между зафиксированными событиями и формирования ответа на запрос оператора с указанием трассировки активации и источников из исходного текста.

Механизм активации (Ignition) отвечает не за непосредственную генерацию ответа, а за распространение активации по релевантному подграфу памяти. При обработке запроса система сначала находит начальные символы, затем активирует связанные элементы, отбирает факты с учётом типа вопроса и ролей и проверяет их зависимость от начальных символов. После этого формируются текст ответа и цитаты. Связь RECALLS означает обращение к сохранённому факту. Сама по себе такая активация не доказывает истинность факта. Концептуально механизм связан с теорией глобального рабочего пространства (Global Workspace Theory).

2.2. Область применения

Предполагаемая область применения — поддержка ретроспективного анализа журналов инцидентов в системах класса цифрового двойника оператора АСУ ТП. Целевой пользователь — инженер-оператор или аналитик, работающий с журналами инцидентов на промышленных объектах.

Класс решаемых задач: ретроспективный анализ аварийных ситуаций, восстановление причинно-следственных связей по неструктурированным и слабоструктурированным текстовым описаниям инцидентов.

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку программы

Задача состоит в построении программной системы, которая хранит знания об инцидентах в структуре ассоциативно-гетерархической памяти, извлекает из текста журналов структурированные факты с привязкой к источнику, выполняет тактовый вывод по релевантному подграфу и сохраняет трассировку, достаточную для восстановления зависимостей активации и источников ответа.

Формальная модель памяти задаётся кортежем $АН = \langle S, C, P, H, L \rangle$.

- S — символы первого порядка и их сенсорные представления R ;
- C — общие знания и понятия;
- P — личные модели и знания агента о конкретных объектах;
- H — история и эпизоды;
- L — ассоциативные связи.

Операции над памятью и механизм активации работают над этой структурой, но не являются её компонентами. В реализации используются типы `SReference`, `MReference`, `SecondOrderSymbol`, `FunctionalSymbol`, `MemoryList`, `ControlTemplate` и `Hypernode`. При этом необходимо различать предикат, гиперсвязь с заполненными ролями (гиперузел) и ассоциативную связь: например, `HAS_STATE` задаёт тип факта и роли гиперузла, тогда как `RECALLS` является направленной ассоциативной связью, используемой для активации.

3.2. Описание применяемых математических методов, допущений и ограничений

В основе механизма активации лежит последовательное обновление состояния элементов памяти. В тексте важно различать три величины: внутреннюю возбудимость x элемента, суммарный входящий импульс z и выход функции активации $f(z, x)$. Они не являются взаимозаменяемыми: z накапливает вход от активных связей, $f(z, x)$ определяет передаваемый дальше результат, а x хранит внутреннее состояние элемента. Ключевые параметры имеют следующие роли:

- `decay_lambda` управляет скоростью затухания;
- `hebbian_eta` — силой изменения весов;
- `rhythm_hz` — частотой ритмоводителя;
- `tick_seconds` — длительностью такта;
- Порог рабочей памяти — границей включения и исключения элементов;
- `initial_life_ticks` — защитой молодых элементов от сборки мусора.

Для интегрирующих профилей используется инженерное правило обновления состояния

$$x_{next} = \text{clip}(x + z) \cdot e^{-\lambda \Delta t} \quad (1)$$

Эта формула фиксирует выбранный способ интеграции входного импульса и затухания в реализации

Функция затухания задаёт экспоненциальное уменьшение внутренней возбудимости. Изменение `decay_lambda` непосредственно влияет на длительность сохранения активации: при большем значении следующее состояние уменьшается быстрее, при меньшем — активация сохраняется дольше.

Правило Хебба применяется при ненулевом значении `hebbian_eta` и изменяет веса связей в зависимости от активности связанных элементов. Для обычной пары связанных элементов используется одно правило обновления, а для гиперсвязи — отдельное правило. Эти случаи нельзя описывать одной формулой. Значение `hebbian_eta = 0` принято в базовой конфигурации, поэтому веса не изменяются между запросами и результаты проще воспроизводить.

Порог рабочей памяти равен 0,2 в базовой конфигурации. Элемент входит в рабочую память при $x > 0,2$ и выходит при $x < 0,2$; при равенстве порогу сохраняется предыдущее состояние членства. Такое правило задаёт гистерезис и не позволяет колебаниям ровно на границе порога постоянно переключать состояние. Значение 0,2 является инженерным компромиссом между доступностью релевантных фактов и попаданием в рабочую память нерелевантных элементов.

ГС формирует кандидатов на удаление среди элементов С/Р/Н, исключая шаблоны управления и узлы в периоде первоначальной защиты. Кандидат удаляется при отсутствии связности с символами S либо при наличии только нулевых весов; затем учитываются зависимости и ссылочная целостность. S служит корнем проверки связности и текущим ГС не удаляется. Режим `preview` не меняет память, режим `commit` применяет удаление.

Молодые элементы защищены параметром `initial_life_ticks`. После истечения заданного числа тактов кандидат может быть удалён только при сохранении ссылочной целостности памяти.

Допущения реализации: все мутации памяти проходят через единый класс-владелец структуры (централизованное изменение состояния), с атомарностью на уровне отдельной операции/факта, а не всего документа. Связи IS-A и FOLLOW проверяются на ацикличность при добавлении; каждый допущенный факт сопровождается объектом привязки к источнику. Иерархическая организация не перестраивается динамически в ходе вывода: активация элементов S, C и P, включая распространение по петлям, не открывает новых классов и не означает полноценного обучения иерархии.

3.3. Описание алгоритма и функционирования программы

3.3.1. Общая архитектура

Система построена по трёхслойной схеме.

1) Слой хранения и мутации (`core.py`) — единственный владелец структуры АГ-памяти: операции добавления, изменения и поиска выполняются через класс `АНMemory`.

2) Слой логики вывода:

- `engine.py` содержит тактовый движок активации и сборщик мусора.
- `dsl.py` — интерпретатор предметно-ориентированного языка запросов.
- `answering.py` выбирает и формулирует ответ на основании ролей и доказательств.

3) Слой ввода-вывода и оркестрации: `main.py` предоставляет HTTP API на базе FastAPI, `ingestion.py` разбирает текст и компилирует извлечённые факты в структуру памяти.

После тактового прогона `main.py` вызывает `engine.evidence_trace`, которая проверяет путь зависимости выбранных узлов от `seed`-символов. При отсутствии такого пути API отказывает в ответе. `ingestion.py` отвечает за разбор, проверку источника и компиляцию фактов.

Вспомогательные модули:

- `models.py` — типизированные модели данных на базе Pydantic;
- `evaluation.py` — внутренние проверки качества;
- `conformance.py` — проверки соответствия формальной модели;
- `persistence.py` — опциональный адаптер для Neo4j.

Воспроизводимость обеспечивается сохранением сырых ответов языковой модели, строгим воспроизведением запуска, хэшированием источников и фиксацией версии кода, что позволяет сравнивать изменения реализации без подмены результата случайным ответом модели.

3.3.2. Алгоритм приёма фактов из текста

Процесс приёма фактов из текста состоит из пяти шагов.

1. Текст документа сегментируется на фрагменты с фиксацией символьных смещений начала и конца каждого фрагмента.

2. Средство извлечения (правильный парсер либо внешний поставщик языковой модели, совместимый с интерфейсом OpenAI API) распознаёт в фрагментах предикаты CAUSE, FOLLOW, IS-A, LOCATED_AT, USES_TOOL, HAS_STATE, OBSERVED. Предикат задаёт тип факта; после заполнения ролей он представляется гиперсвязью (гиперузлом), а отдельные ассоциативные связи, например RECALLS, используются механизмом активации. Результат — список фактов-кандидатов.

3. Для каждого кандидата формируется объект привязки к источнику: идентификатор документа и фрагмента, начальное и конечное смещение, точный текст фрагмента, хэш содержимого, идентификатор запуска парсера и уровень уверенности.

4. Оператор или автоматическое правило допускает факт в память либо отклоняет его.

5. Компилятор валидирует и дедуплицирует допущенные факты, затем фиксирует каждый в памяти: создаёт элементы, связи и управляющие шаблоны. Операция атомарна для конкретного

факта — остальные факты входного текста не принимаются вместе с ним автоматически.

Актуальный набор шаблонов управления берётся из `TEMPLATE_ROLES`; он содержит 14 типов шаблонов и 10 различных имён ролей: `CAUSE`, `HOW-TO`, `LOCATION`, `OBJECT`, `PURPOSE`, `RESULT`, `STATE`, `SUBJECT`, `TIME`, `TOOL`. Модельному поставщику предлагаются 13 типов шаблонов, поскольку `IN_SCOPE` извлекается детерминированно из явного утверждения об области в именованном разделе, без обращения к модели. Роль `RESULT` принадлежит одноимённому шаблону. При нормализации `CAUSE.RESULT` отображается в `OBJECT`, что не образует отдельного типа роли (источник факта хранится отдельно). Число и состав ролей — характеристика текущей реализации, а не постоянное свойство архитектуры.

3.3.3. Алгоритм активации

При поступлении запроса система ищет начальные символы по тексту вопроса, формирует неизменяемый снимок релевантного подграфа и запускает тактовый цикл. После активации `answering.py` отбирает факты с учётом типа вопроса и ролей, проверяет их зависимость от начальных символов и формирует ответ с цитатами. Тактовая трасса и `source offsets` показывают, какой путь активации привёл к выбранному гиперузлу и из какого текстового фрагмента был составлен факт. Наличие такой трассы не подтверждает семантическую корректность извлечения и не заменяет отдельную экспертную проверку ответа.

На каждом такте: для обычных связей и отдельно для гиперузлов вычисляются входящие импульсы — они суммируются, обновляется внутреннее состояние x по выбранному профилю активации и затухания, определяется рабочая память по правилу с разрешением граничного случая (вход при $x > \text{порога}$, выход при $x < \text{порога}$, при равенстве — прежнее членство; это правило разрешения равенства, а не доказательство широкой зоны гистерезиса).

Максимальное число тактов ограничено конфигурацией. Если найдены факты, зависящие от начальных символов и удовлетворяющие типу вопроса, система формирует ответ с текстовой формулировкой, трассой зависимостей активации, трассировкой тактов и ссылками на исходные фрагменты. Поле `minimal_path` сохраняется в API по историческим причинам, но не гарантирует кратчайший логический путь — оно восстанавливает замыкание зависимостей активации; полная трасса подтверждает путь вычисления и источник, но не исправляет ошибку извлечения факта и не делает факт истинным.

Каждый такт фиксируется в объекте трассировки: тип и значение импульса, предыдущий и следующий уровень активации, факт входа/выхода из порога, изменения весов связей, ссылка на родительскую трассировку. Наличие источника, корректность извлечения факта, его достижимость от вопроса и корректность самого рассуждения — разные свойства, которые оцениваются отдельно.

Предусмотрены три профиля активации: `literal_2026`, `integrated_v1`, `directed_v1`. Профиль `literal_2026` — буквальное прочтение формул монографии в принятой в проекте интерпретации, а не утверждение о доказанном полном соответствии реализации формализму монографии.

3.3.4. Сборщик мусора

Сборщик мусора работает в двухфазном режиме. Он рассматривает элементы, относящиеся к разделу *S* и удовлетворяющие условиям отсутствия ненулевых весов и истечения периода защиты. Молодые элементы не удаляются до окончания *initial_life_ticks*. В режиме предпросмотра система вычисляет кандидатов без изменения памяти; в режиме фиксации выполняется удаление с сохранением ссылочной целостности.

3.3.5. Интерпретатор предметно-ориентированного языка

Интерпретатор поддерживает операции чтения: поиск абстрактных символов, поиск гиперузлов, поиск ролей, обход связей на заданную глубину и пересечение множеств. Через этот интерфейс доступны две минимальные операции мутации: добавление связи *addLink* с указанием типа, источника, приёмника и веса и добавление элемента *addElement*.

3.3.6. Взаимодействие с внешними системами

Взаимодействие с внешним поставщиком языковой модели осуществляется по протоколу HTTP через адаптер, совместимый с интерфейсом OpenAI API. Настройка выполняется через переменные окружения: адрес конечной точки, имя модели и ключ доступа. При отсутствии настройки система автоматически переключается на правилый парсер. Для воспроизводимых сравнений сохраняются сырые ответы модели и данные, необходимые для воспроизведения.

Взаимодействие с графовой базой данных Neo4j реализовано через опциональный адаптер и требует установки отдельной зависимости. По умолчанию данные хранятся в памяти процесса.

Взаимодействие с клиентскими приложениями осуществляется через HTTP REST API. Формат запросов и ответов — JSON. В текущем проекте зафиксировано 24 явных GET/POST-маршрута, включая / и /health; число маршрутов следует относить к конкретной версии кода. Маршруты покрывают приём фактов из текста, запросы, трассировку, сборщик мусора, статистику памяти, экспорт, проверку соответствия и демонстрационные сценарии.

3.4. Обоснование выбора метода организации входных и выходных данных

Входные данные поступают в систему в двух формах. Неструктурированный текст журнала инцидента передаётся в теле запроса и сегментируется на фрагменты с сохранением символьных смещений и хэша содержимого для обеспечения обратной прослеживаемости. Структурированный запрос оператора передаётся в формате JSON и содержит параметры конфигурации вывода.

Выходные данные формируются в формате JSON и включают текст ответа, список идентификаторов элементов рабочей памяти, трассу зависимостей активации, полную трассировку тактов и массив привязок к исходному тексту. Поле *minimal_path* в API не следует интерпретировать

как гарантированно кратчайший путь.

Выбор формата JSON обусловлен нативной поддержкой в используемой фреймворке без дополнительных преобразований, возможностью передачи вложенных структур без потери типизации и совместимостью с веб-интерфейсом, реализованным на HTML, CSS и JavaScript.

Внутреннее представление данных реализовано в виде типизированных моделей на базе библиотеки Pydantic, обеспечивающих валидацию на уровне структуры. В ANMemory память представлена индексами symbols, sections, links и templates. Доступ к отдельному UID через соответствующий индекс обычно имеет константную сложность. Операции обхода и поиска по значениям могут требовать прохода по коллекции. Хранение в процессе исключает обращение к внешней СУБД в базовом режиме.

3.5. Описание и обоснование выбора состава технических и программных средств

В качестве основного языка реализации выбран Python версии не ниже 3.12, что обеспечивает поддержку современной системы типизации и совместимость с библиотекой валидации данных.

Фреймворк FastAPI выбран в качестве основы HTTP API благодаря автоматической генерации спецификации интерфейса и поддержке асинхронной обработки запросов. Число маршрутов HTTP API приведено в п. 3.3.6.

Библиотека Pydantic обеспечивает типизацию и валидацию всех моделей данных, включая семь типов элементов памяти, связи, конфигурацию вывода и трассировку. Режим неизменяемых объектов применяется для защиты целостности данных.

Сервер Uvicorn используется в качестве ASGI-сервера для запуска приложения.

Графовая база данных Neo4j подключается опционально при необходимости персистентного хранения. В базовой конфигурации данные хранятся в памяти процесса, что упрощает развёртывание и устраняет внешние зависимости.

Векторный поиск на базе библиотек sentence-transformers и FAISS/ChromaDB заявлен в архитектурной спецификации, но в основном АГ-агенте на момент сдачи не реализован; для самого механизма АГ-памяти он не обязателен, поскольку причинно-следственные связи представлены типизированными рёбрами графа.

Требования к М4 и М5 заданы в разделе 7 задания хакатона. М4 требует контролируемого парного сравнения с Vanilla LLM + RAG на том же корпусе и той же модели с расчётом *explainability* и *hallucination*. М5 требует двух классов моделей, SLM до 8 млрд параметров и коммерческой LLM, с расчётом RobustnessGain. В настоящей версии записки имеются только предварительные локальные результаты, не итоговые значения М4/М5.

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

Ожидаемое технико-экономическое преимущество решения перед embedding-based RAG — не в объёме извлечённых данных, а в верифицируемости ответа: каждый повлиявший на ответ факт прослеживается до исходного фрагмента текста через полную трассировку активации и привязку к источнику.

Архитектура предусматривает проверяемую техническую прослеживаемость: выбранные гиперузлы связываются с тактовой трассой и точными фрагментами исходного текста. Это подтверждает происхождение и путь активации данных, но не истинность извлечённого утверждения и не семантическую правильность ответа. Преимущество перед RAG по качеству, объяснимости или доле неподтверждённых ответов требует парного эксперимента и не выводится из наличия трассы.

Количественное сравнение с RAG (M4–M5) на момент сдачи не завершено: контрольный RAG-агент не доведён до состояния, пригодного для контролируемого сравнения, а протокол оценки, корпус и набор вопросов зафиксированы лишь частично.

Предварительный сохранённый парный пилот Qwen на общей выборке 40 вопросов проводился: по последующей неслепой смысловой разметке АН дал 6 правильных содержательных ответов и 16 правильных отказов (22/40), RAG — 18 правильных содержательных ответов и 16 правильных отказов (34/40). Это результаты прежних сохранённых ответов, не текущего V11 и не официальное M4. На V11 локальный непарный строгий QA по записанным предсказаниям составил 21/40: 8 правильных содержательных ответов и 13 правильных отказов; старый регрессионный набор — 35/35. Эти оценки имеют разные протоколы, поэтому сравнивать 21/40 с RAG 34/40 как текущий А/В-тест нельзя. Коммерческая ветка M5 и итоговые M4/M5 не измерены.

5. ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ

- Душкин Р. В. «Монография по ассоциативно-гетерархической памяти».
- Global Workspace Theory (теория глобального рабочего пространства) — концептуальная основа механизма активации.
- Задание хакатона «Воспламенение 1.0»